Stockholm University

Department of Meteorology

Numerical Methods in Atmospheric Sciences and Oceanography (MO8007)

Some short notes on the 2D Shallow water equations

April 22, 2026

Author: Fredrik Bergelv (fredrik.bergelv@live.se)

Supervisor: Hanna Winge

1 Background

The 2D shallow water equations (SWE) in their linearized form are given by

$$\frac{\partial u}{\partial t} - fv = -g\frac{\partial h}{\partial x}, \tag{1}$$

$$\frac{\partial v}{\partial t} + fu = -g\frac{\partial h}{\partial y}, \tag{2}$$

$$\frac{\partial h}{\partial t} + H_0\left(\frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}\right) = 0, \tag{3}$$

where u and v are the horizontal velocity components, h is the surface height perturbation, f is the Coriolis parameter, g is the gravitational acceleration, and H_0 is the mean fluid depth.

These equations can be discretized using a Leapfrog scheme, which is centered in both time and space. On an Arakawa C-grid, this yields

$$u_{i,j}^{n+1} = u_{i,j}^{n-1} + 2\Delta t\left[-g\frac{h_{i+1,j}^n - h_{i,j}^n}{\Delta x} + \frac{f}{4}\left(v_{i,j}^n + v_{i+1,j}^n + v_{i+1,j-1}^n + v_{i,j-1}^n\right)\right], \tag{4}$$

$$v_{i,j}^{n+1} = v_{i,j}^{n-1} + 2\Delta t\left[-g\frac{h_{i,j+1}^n - h_{i,j}^n}{\Delta y} - \frac{f}{4}\left(u_{i,j}^n + u_{i,j+1}^n + u_{i-1,j+1}^n + u_{i-1,j}^n\right)\right], \tag{5}$$

$$h_{i,j}^{n+1} = h_{i,j}^{n-1} - 2H_0\Delta t\left[\frac{u_{i,j}^n - u_{i-1,j}^n}{\Delta x} + \frac{v_{i,j}^n - v_{i,j-1}^n}{\Delta y}\right]. \tag{6}$$

The Courant number is defined as

$$\mu = \frac{\sqrt{gH_0}\,\Delta t}{\sqrt{\Delta x^2 + \Delta y^2}}, \tag{7}$$

and a stable solution requires $\mu < 0.5$ for the staggered (C-grid) Leapfrog scheme.

Since the Leapfrog method requires two previous time levels, an initial step must be computed using a Forward Euler scheme:

$$u_{i,j}^1 = u_{i,j}^0 + \Delta t\left[-g\frac{h_{i+1,j}^0 - h_{i,j}^0}{\Delta x} + \frac{f}{4}\left(v_{i,j}^0 + v_{i+1,j}^0 + v_{i+1,j-1}^0 + v_{i,j-1}^0\right)\right], \tag{8}$$

$$v_{i,j}^1 = v_{i,j}^0 + \Delta t\left[-g\frac{h_{i,j+1}^0 - h_{i,j}^0}{\Delta y} - \frac{f}{4}\left(u_{i,j}^0 + u_{i,j+1}^0 + u_{i-1,j+1}^0 + u_{i-1,j}^0\right)\right], \tag{9}$$

$$h_{i,j}^1 = h_{i,j}^0 - H_0\Delta t\left[\frac{u_{i,j}^0 - u_{i-1,j}^0}{\Delta x} + \frac{v_{i,j}^0 - v_{i,j-1}^0}{\Delta y}\right]. \tag{10}$$

The Leapfrog scheme suffers from a computational mode, which manifests as a time-oscillating numerical artifact. A common way to suppress this mode is to apply the Robert–Asselin filter:

$$u^n \leftarrow u^n + \gamma\left(u^{n-1} - 2u^n + u^{n+1}\right), \tag{11}$$

and similarly for v and h, where γ is a small filtering parameter. The filter modifies the stability condition, which for the staggered Leapfrog scheme becomes approximately

$$\mu + \gamma \leq 0.5. \tag{12}$$

To quantify numerical accuracy, a relative error measure can be defined as

$$\text{error}^n = \frac{\|\mathbf{h}_{\text{numerical}}^n - \mathbf{h}_{\text{reference}}^n\|}{\|\mathbf{h}_{\text{reference}}^n\|}, \tag{13}$$

where $\|\cdot\|$ denotes an appropriate norm and the reference solution may be the initial condition or an analytical solution.

2 Methodology

A localized perturbation in the free surface height was initialized as a circular bump centered in the domain. The initial condition is a localized Gaussian perturbation centered in the domain:

$$h(x, y, 0) = 10 \exp\left(-\frac{(x - x_c)^2 + (y - y_c)^2}{0.01\, L_x^2}\right), \tag{14}$$

where x_c and y_c was chosen to be at the domain center.

Solid wall boundary conditions were applied at all domain boundaries. This implies that the velocity components vanish at the boundaries as

$$u = 0 \quad \text{at } x = 0,\, L_x, \qquad v = 0 \quad \text{at } y = 0,\, L_y, \tag{15}$$

which leads to reflection of waves at the domain edges.

The computational domain was discretized using a uniform grid with the following parameters:

$$N_x = 100, \qquad N_y = 100, \tag{16}$$
$$L_x = 100000\,\text{m}, \qquad L_y = 100000\,\text{m}, \tag{17}$$
$$L_t = 1000\,\text{s}. \tag{18}$$

Other useful parameters were set to $f = 10^{-4}$ s^{-1}, $g = 10$ m s^{-2}, $H_0 = 1000$ m, $\mu = 0.45$, and $\gamma = 0.05$.

Model output was saved every 20 time steps in order to reduce storage requirements while still resolving the temporal evolution of the solution. The total energy and volume were computed and stored at every time step as

$$V = \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} h_{i,j}\, \Delta x\, \Delta y, \tag{19}$$

$$E_P = \frac{1}{2} g \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} h_{i,j}^2\, \Delta x\, \Delta y, \tag{20}$$

$$E_K = \frac{1}{2} H_0 \sum_{i=1}^{N_x} \sum_{j=1}^{N_y} \left(u_{i,j}^2 + v_{i,j}^2\right) \Delta x\, \Delta y, \tag{21}$$

where V is the total volume, E_P is the potential energy, and E_K is the kinetic energy of the system.

The Fortran code used to simulate this scheme can be seen in Appendix A.

3 Analysis

In Figure 1, the evolution of the wave field originating from the initial perturbation can be observed. The solution exhibits radial outward wave propagation, while no clear Coriolis-induced deflection is visible at this timescale. After approximately 12 minutes, the waves reach the domain boundaries, where they are reflected due to the imposed solid wall conditions. Another notable feature is that the waves propagate relatively quickly across the domain, which however is consistent with the chosen parameter values.

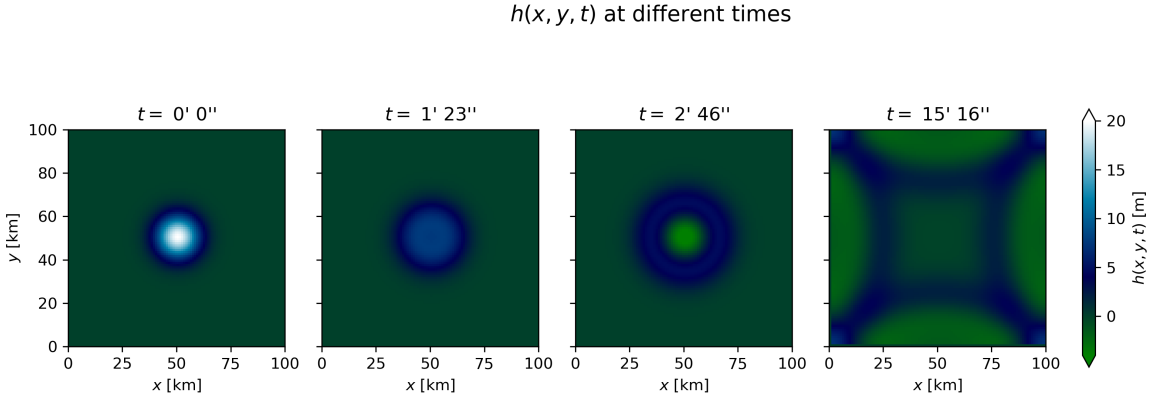

Figure 1: Time evolution of the surface height $h(x, y, t)$ showing wave propagation and boundary reflections.

In Figure 2, the evolution of total energy and volume is shown for the numerical model. The volume remains nearly constant throughout the simulation, indicating good mass conservation. However, the total energy decreases by approximately 25%, suggesting the presence of numerical dissipation. The time required for the wave to reach the boundary is consistent with the expected propagation speed, occurring at approximately 12 minutes (around 700 seconds), as also visible in the energy evolution plot.

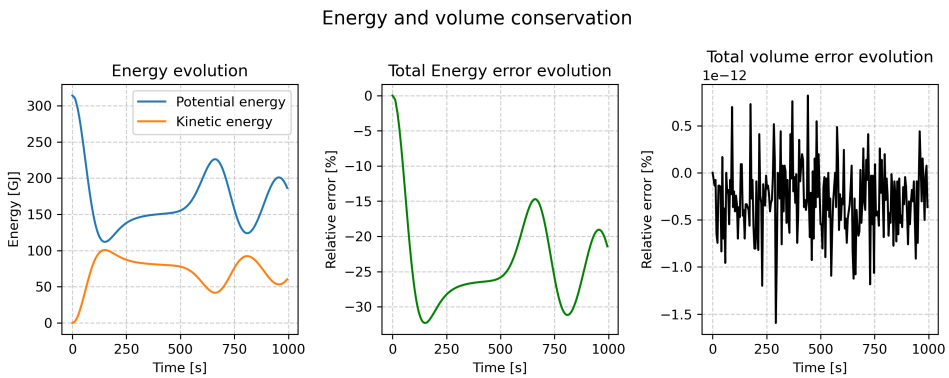

Figure 2: Time evolution of potential, kinetic, and total energy, showing approximate conservation with minor numerical dissipation.

The large discrepancy in total energy should be further investigated. The influence of the Robert–Asselin filter was examined, but it does not appear to be the primary source of the energy loss. Similarly, variations in the initial conditions do not significantly affect the observed behaviour. This suggests that the main source of error is likely the initial Forward Euler step used to start the Leapfrog scheme.

This is also consistent with the observation that the largest drop in accuracy occurs at the beginning of the simulation, after which the energy evolution becomes more stable. This indicates that the initial time-stepping procedure may introduce a transient error that is not subsequently amplified by the Leapfrog integration.

A The code

```
PROGRAM Cgrid

implicit none

! Namelist parameters
real :: CFL, f, dx, dy, g, H0, Lx, Ly, Lt, alpha
namelist /params/ CFL, f, dx, dy, g, H0, Lx, Ly, Lt, alpha

! VOLariables
real :: dt
integer :: Nx, Ny, Nt
real :: xc, yc, r, r2

integer :: i, j, n, ip, im, jp, jm
real :: x, y
real(8), allocatable :: h(:,:,:), u(:,:,:), v(:,:,:)
character(len=100) :: filename
real, parameter :: pi = 4.0 * atan(1.0)
integer :: savefreq = 20
real(8) :: VOL, EP, EK

! Read namelist
open(10, file="2D-SWE/input.nml", status="old")
read(10, nml=params)
close(10)

! Compute derived values
dt = CFL / ( sqrt(g*H0) * sqrt( (1/dx**2) + (1/dy**2) ) )
Nx = int(Lx/dx)
Ny = int(Ly/dy)
Nt = int(Lt/dt)

! Allocate arrays
allocate(h(Nx,Ny,3), u(Nx,Ny,3), v(Nx,Ny,3))

! Open file for saving
filename = "2D-SWE/h_array_wall.txt"
open(10, file=filename, status="replace")

open(20, file="2D-SWE/energy.txt", status="replace")


! ----------------------------
! The actual code
! ----------------------------

! Initialize (n = 1)
u(:,:,1) = 0.0
v(:,:,1) = 0.0


DO j = 1, Ny
    do i = 1, Nx
        x = (i-1)*dx
        y = (j-1)*dy
```

```
        xc = 0.5*Lx
        yc = 0.5*Ly
        r  = sqrt( (x - xc)**2 + (y - yc)**2 )

        r2 = (x - xc)**2 + (y - yc)**2

        if (r < 0.3*Lx) then
            h(i,j,1) = 10.0 * exp(-r2 / (0.01*Lx*Lx))
        else
            h(i,j,1) = 0.0
        end if
    end do
END DO

write(10, *) h(:,:,1) ! Save

call compute_energy(h(:,:,1), u(:,:,1), v(:,:,1), dx, dy, H0, g, VOL, EP, EK)
write(20, *) n*dt, VOL, EP, EK


! First step (Forward Euler)
DO j = 1, Ny
    DO i = 1, Nx

        ! WALL BOUNDARIES
        ip = i + 1
        im = i - 1
        jp = j + 1
        jm = j - 1

        if (i == Nx) ip = i
        if (i == 1)  im = i
        if (j == Ny) jp = j
        if (j == 1)  jm = j

        ! Do the Euler forward
        h(i,j,2) = h(i,j,1) - H0*dt * ((u(i,j,1)-u(im,j,1))/dx + (v(i,j,1)-v(i,jm,1))/dy)

        u(i,j,2) = u(i,j,1) + dt * ( &
            -g*(h(ip,j,1)-h(i,j,1))/dx + &
             f*(v(i,j,1)+v(ip,j,1)+v(ip,jm,1)+v(i,jm,1))/4 )

        v(i,j,2) = v(i,j,1) + dt * ( &
            -g*(h(i,jp,1)-h(i,j,1))/dy - &
             f*(u(i,j,1)+u(i,jp,1)+u(im,jp,1)+u(im,j,1))/4 )

    END DO
END DO

!write(10, *) h(:,:,2) ! Save


! Enforce wall BC
DO j = 1, Ny
    DO i = 1, Nx

        if (i == 1 .or. i == Nx) then
```

```
            u(i,j,1) = 0.0
            u(i,j,2) = 0.0
            u(i,j,3) = 0.0
        end if

        if (j == 1 .or. j == Ny) then
            v(i,j,1) = 0.0
            v(i,j,2) = 0.0
            v(i,j,3) = 0.0
        end if

    END DO
END DO


! Leapfrog time stepping
DO n = 2, Nt-1

    DO j = 1, Ny
        DO i = 1, Nx

            ! WALL BOUNDARIES
            ip = i + 1
            im = i - 1
            jp = j + 1
            jm = j - 1

            if (i == Nx) ip = i
            if (i == 1)  im = i
            if (j == Ny) jp = j
            if (j == 1)  jm = j

            ! Leapfrog scheme
            h(i,j,3) = h(i,j,1) - 2*H0*dt * ((u(i,j,2)-u(im,j,2))/dx + (v(i,j,2)-v(i,jm,2))/dy)

            u(i,j,3) = u(i,j,1) + 2*dt * ( &
                -g*(h(ip,j,2)-h(i,j,2))/dx + &
                 f*(v(i,j,2)+v(ip,j,2)+v(ip,jm,2)+v(i,jm,2))/4 )

            v(i,j,3) = v(i,j,1) + 2*dt * ( &
                -g*(h(i,jp,2)-h(i,j,2))/dy - &
                 f*(u(i,j,2)+u(i,jp,2)+u(im,jp,2)+u(im,j,2))/4 )

        END DO
    END DO

    ! Asselin filter
    h(:,:,2) = h(:,:,2) + alpha * (h(:,:,1) - 2*h(:,:,2) + h(:,:,3))
    u(:,:,2) = u(:,:,2) + alpha * (u(:,:,1) - 2*u(:,:,2) + u(:,:,3))
    v(:,:,2) = v(:,:,2) + alpha * (v(:,:,1) - 2*v(:,:,2) + v(:,:,3))

    ! Shift time levels
    h(:,:,1) = h(:,:,2)
    h(:,:,2) = h(:,:,3)

    u(:,:,1) = u(:,:,2)
    u(:,:,2) = u(:,:,3)
```

```
    v(:,:,1) = v(:,:,2)
    v(:,:,2) = v(:,:,3)



    ! Enforce wall BC
    DO j = 1, Ny
        DO i = 1, Nx
            if (i == 1 .or. i == Nx) then
                u(i,j,1:3) = 0.0
            end if
            if (j == 1 .or. j == Ny) then
                v(i,j,1:3) = 0.0
            end if
        END DO
    END DO

    if (mod(n,savefreq) == 0) then
        write(10, *) h(:,:,2)

    end if

    call compute_energy(h(:,:,2), u(:,:,2), v(:,:,2), dx, dy, H0, g, VOL, EP, EK)
        write(20, *) n*dt, VOL, EP, EK

    !print*, "t = ", n*dt

END DO

close(10)
close(20)



print *, "Simulation completed, saved as: ", filename

    write(*,'(A,I0)') "Nx = ", Nx
    write(*,'(A,I0)') "Ny = ", Ny
    write(*,'(A,I0)') "Lt = ", int(Lt)
    write(*,'(A,I0)') "Lx = ", int(Lx)
    write(*,'(A,I0)') "Ly = ", int(Ly)
    write(*,'(A,I0)') "stride = ", savefreq





contains

subroutine compute_energy(hin, uin, vin, dx_s, dy_s, H0_s, g_s, VOL_s, EP_s, EK_s)

    implicit none

    real, intent(in) :: dx_s, dy_s, H0_s, g_s

    real(8), intent(in) :: hin(Nx,Ny), uin(Nx,Ny), vin(Nx,Ny)

    real(8), intent(out) :: VOL_s, EP_s, EK_s
```

```
        integer :: i_s, j_s

        VOL_s  = 0.0d0
        EP_s = 0.0d0
        EK_s = 0.0d0

        !print*, maxval(hin(:,:))

        do j_s = 1, Ny
            do i_s = 1, Nx

                VOL_s  = VOL_s  + hin(i_s,j_s) * dx_s * dy_s
                EP_s = EP_s + 0.5d0 * g_s  * hin(i_s,j_s)**2 * dx_s * dy_s
                EK_s = EK_s + 0.5d0 * H0_s * (uin(i_s,j_s)**2 + vin(i,j)**2) * dx_s * dy_s



            end do
        end do

    end subroutine compute_energy

END PROGRAM Cgrid
```